1. **Where would you rate yourself on (LLM, Deep Learning, AI, ML)? A, B, C [A = can code independently; B = can code under supervision; C = have little or no understanding]**

B= can code under supervision; understands and knows the usage of library methods and understands complex codes

2. What are the key architectural components to create a chatbot based on an LLM? Please explain the approach at a high level

Language plays a fundamental role in facilitating communication and self-expression for humans and their interaction with machines

Key Architectural Components of an LLM-Based Chatbot

An LLM-based chatbot enables natural-language communication between humans and machines. Its architecture consists of several components that work together to understand user queries, retrieve relevant information, generate responses, and deliver the output.

1. Data Preprocessing and Tokenization

Training data is prepared through quality filtering, deduplication, and privacy reduction. Tokenization converts text into smaller units called tokens that the model can process.

2. Transformer Architecture

The Transformer is the core architecture of most modern LLMs. Its major components include token embeddings, positional encodings, self-attention layers, feed-forward neural networks, activation functions, and layer normalization. Together, these components help the model process context and generate text.

3. Model Training and Fine-Tuning

A pretrained LLM learns language patterns from large datasets. Fine-tuning adapts it to specific tasks using task-specific or instruction-formatted data. Human-feedback-based alignment can further improve the model's responses and behavior.

4. Prompting and Utilization

Prompts provide instructions and context to guide the model's responses. Techniques include zero-shot prompting, few-shot prompting, and prompt tuning. These methods help adapt the model's behavior to different tasks**.**

5. Backend and API Integration

A backend connects the chatbot interface to the LLM. An API receives user queries, manages requests, calls the model, and returns generated responses. Frameworks such as FastAPI can be used to implement this component.

6. Retrieval-Augmented Generation (RAG)

RAG allows the chatbot to retrieve relevant information from external documents or databases before generating an answer. This can improve factual grounding and provide source references when supported by the retrieval system.

7. Tool and Multimodal Integration

The chatbot can be extended to use external tools, such as databases and APIs, or process inputs such as images and audio. When tools are needed, the system can select a tool, execute it, inspect the result, and use that result to formulate a response**.**

8. Output Generation and Delivery

The LLM generates a response, which may undergo validation, safety checks, and formatting before the backend returns it to the user interface.

3. Please explain vector databases. If you were to select a vector database for a hypothetical problem (you may define the problem), which one would you choose, and why?

Vectors are numerical representations of data that capture useful information about text, images, audio, and video. These representations are called vector embeddings and are widely used in artificial intelligence applications such as large language models (LLMs), question-answering systems, image recognition, and recommendation systems.

A vector database is a database designed to store and search these embeddings efficiently. Traditional databases are mainly used to store structured information and retrieve records based on exact values. However, when we need to find information based on meaning or similarity, vector databases are more suitable.

For example, if a user searches for “How can I recover my account?”, a vector database may retrieve a document about resetting a forgotten password, even if the wording is different.

The main benefits of vector databases are efficient similarity search, support for embeddings generated from different types of data, and the ability to handle growing datasets through suitable indexing and scaling methods.

The process starts with collecting data, such as documents or images. If the data consists of long documents, it is divided into smaller chunks. An embedding model then converts each chunk into a numerical vector, which is stored in the vector database along with its text and relevant metadata.

When a user submits a query, the same embedding model converts it into a vector. The database compares this vector with the stored vectors using a similarity measure, such as cosine similarity, and retrieves the most relevant results.

These results can then be passed to an LLM to generate an answer. This approach is commonly used in Retrieval-Augmented Generation (RAG) systems.Vector databases are used in several AI applications:

- **Semantic search:** Finds relevant information based on meaning rather than exact keywords.
- **RAG chatbots:** Retrieves information from documents and provides context to an LLM for answering questions.
- **Recommendation systems:** Suggests products, movies, or other items based on similarities in user preferences or item features.
- **Image search:** Finds visually or semantically similar images using image embeddings.
- **Audio and multimedia search:** Retrieves related media using appropriate embedding models.
- **Duplicate detection:** Identifies similar documents, images, or other content.

Suppose I want to build a chatbot that helps students find information from technical books, lecture notes, and PDF documents. Instead of searching for exact keywords, the chatbot should understand the meaning of a question, retrieve relevant passages, and use an LLM to generate an answer.

For this application, I would choose **ChromaDB**.

I would choose ChromaDB because it is easy to use with Python and is suitable for building RAG applications. It supports vector similarity search, metadata filtering, and persistent storage. These features would help me organize document chunks by book title or subject and retrieve relevant information when students ask questions.

For implementation, I would extract text from the documents, split it into smaller chunks, and generate embeddings for each chunk. I would store the embeddings and associated document information in ChromaDB. When a student asks a question, I would generate an embedding for the query, retrieve the most relevant passages, and pass them to the LLM to generate an answer based on the retrieved information.

Other options include FAISS, which is useful when efficient vector similarity search is the main requirement, and Qdrant, which is worth considering for applications that need a dedicated vector database with advanced filtering and flexible deployment options.